8-2 交换排序

1.冒泡排序

- 基本思想
  - 从前往后(反过来也行)依次比较相邻元素，如果逆序则交换，不断循环到有序
- 其他概念
  - 对所有元素进行过一遍处理称为一趟
  - 冒泡排序可以进行优化
  - 如果一趟中未发生元素交换，可以直接结束排序
  - 是稳定的排序算法
- 代码表示

传入待排序数组

```
void BubbleSort(SqList &L)
{ // 对顺序表 L 进行冒泡排序
    m=L.length-1;
    flag=1;
    //如果本趟排序没有发生交换，则不会执行下一趟排序
    while((m>0)&&(flag==1)){
        flag=0; //flag用来标记某一趟排序是否发生交换
        for(j=1;j<=m;j++){
            if(L[j]>L[j+1]){
                flag=1; //表示本趟排序发生了交换
                t=L[j];
                L[j]=L[j+1];
                L[j+1]=t;
                }
            } //for
        --m;
        } //while
} //BubbleSort
```

2.快速排序

- 基本思想
  - 选择一个元素作为枢纽（选择表中第一个元素）
  - 让枢纽左边存储小于枢纽的元素，右边存储大于枢纽的元素
  - 具体做法
    - 表左右侧设立low,high指针，初始low=1;high=L.length
    - high指针往左（小）走，碰到比枢纽元素小的就把元素放到low位置，然后操作low指针
    - low指针往右（大）走，碰到比枢纽元素大的就把元素放到high位置，然后操作high指针
- 代码表示

```
void QSort(SqList &L,int low,int high)
{//调用前置初值：low=1; high=L.length;
// 对顺序表 L 中的子表 L[low..high] 进行快速排序
if(low<high)
{ //长度大于1
    pivotloc=Partition(L, low, high);
    //将L[low..high]一分为二，pivotloc是枢轴位置
    QSort(L, low, pivotloc-1); //对左子表递归排序
    QSort(L, pivotloc+1, high); //对右子表递归排序
}
```

```
int Partition(SqList &L, int low, int high)
{ //对顺序表 L 中的子表 L[low..high] 进行一趟排序，返回枢轴位置
   L[0] = L[low]; // 用子表的第一个记录作为枢轴记录
   pivotkey = L[low]; // 枢轴记录关键字保存在 pivotkey中
   while (low < high){
        while (low < high && L[high] >= pivotkey) --high;
        L[low] = L[high];//将比枢轴记录小的记录移到低端
        while (low < high && L[low] <= pivotkey) ++low;
        L[high] = L[low];//将比枢轴记录大的记录移到高端
        } //while
   L[low] = L[0]; //枢轴记录到位
   return low; //返回枢轴位置
}
```

by joshuji 由 幕布 发布